Train an Intelligent Database with ChatGPT

Yifeng Jiang*
Xi'an Jiaotong-Liverpool University
Suzhou, Jiangsu, China
1608305980@qq.com

Abstract

The integration of AI into database management has the potential to reduce the complexities of SQL, making databases more accessible to non-technical users. Traditional database systems often require users to possess specialized knowledge in SQL, which creates significant barriers for non-technical professionals who need to query or manipulate data. This issue is particularly evident in industries such as marketing and finance, where professionals are often required to interact with databases but lack the necessary coding expertise. This research focuses on developing a Java-based system that utilizes ChatGPT to convert natural language into SQL commands, allowing non-technical users to interact with databases in a more intuitive and user-friendly manner. The primary research question addresses how AI, specifically ChatGPT, can bridge the gap between natural language and complex SQL syntax, enabling seamless database interactions without requiring users to learn SQL. The methodology involves configuring a MySQL environment, integrating the OpenAI API, and iteratively refining the system to handle basic database operations such as querying, inserting, and deleting data. The data for this study is sourced from a pre-existing MySQL database, which includes various sample datasets commonly used for testing database operations. The database contains multiple tables with different types of data, allowing the system to perform a variety of SQL operations. The results show that this system successfully lowers the barriers to database access, significantly improving the ease of use for non-technical users. However, limitations remain in handling more complex SQL queries and occasional inaccuracies in generated SQL commands. This paper concludes that AI-driven tools, like ChatGPT, have great potential to enhance database usability. Future work will focus on improving the accuracy of generated SQL commands, expanding the system's ability to handle complex queries, and refining the user interface to improve user experience.

CCS Concepts

• **Information systems** → Data management systems; Query Languages; Relational database query languages; Structured Query Language.

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
AIBDF 2024, Ganzhou, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1086-5/2024/12
https://doi.org/10.1145/3718491.3718667

Keywords

ChatGPT, Database, Intelligence, MySQL, User-friendly interface

ACM Reference Format:
Yifeng Jiang. 2024. Train an Intelligent Database with ChatGPT. In *The 4th Asia-Pacific Artificial Intelligence and Big Data Forum (AIBDF 2024), December 27–29, 2024, Ganzhou, China.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3718491.3718667

1 Introduction

The rapid development of large artificial intelligence language models (LLMs) in recent years, especially the emergence of models such as ChatGPT, has greatly enhanced the capabilities of natural language processing (NLP). These models are trained using large datasets and can understand and generate complex human language text, demonstrating strong potential in various fields. The widespread adoption of LLM has not only brought breakthroughs to dialogue systems, but also to tasks such as text generation, language translation, and information extraction [1-2]. LLM is capable of capturing deep linguistic structures and semantic features, thereby driving a paradigm shift in NLP technology. By leveraging natural language processing, users can execute database queries using natural language, which greatly reduces the technical threshold and improves the efficiency and flexibility of data retrieval [3].

Despite these advancements, current database management systems (DBMS) continue to present significant challenges for non-technical users. Traditional DBMS require users to possess specialized knowledge of SQL syntax, which acts as a barrier for many professionals who need to interact with data but lack technical expertise. For instance, in industries such as marketing and finance, professionals often need access to data for decision-making but struggle with the steep learning curve of SQL. This disconnect between the technical complexity of SQL and the user-friendly needs of non-technical professionals highlights the importance of developing intuitive systems that bridge this gap. Existing solutions, while effective in some technical scenarios, often fail to provide a seamless and accessible interface for these users, making the need for innovation in this domain more pressing. Furthermore, the complexity of SQL syntax itself often leads to errors, inefficiencies, and a reliance on technical staff for even basic database operations. These challenges limit the accessibility of databases, reduce productivity, and create silos between technical and non-technical teams within organizations. The need for an intuitive interface that lowers the technical barrier to database management is increasingly urgent in a data-driven world [4-7].

Therefore, this paper focuses on training and developing an intelligent database system powered by ChatGPT. The core objective is to utilize ChatGPT to convert natural language commands into SQL statements, enabling users to interact with databases without

requiring SQL expertise. Specifically, the study involves configuring a MySQL environment, integrating the OpenAI API, and iteratively developing a system capable of performing basic database operations such as querying, inserting, and deleting data. By lowering the barriers to database access, the proposed system aims to address the limitations of traditional DBMS and pave the way for more inclusive data management practices. This approach not only reduces the technical threshold for database interaction but also empowers non-technical professionals to access and manage data more effectively. By automating the conversion of natural language to SQL, this research has the potential to democratize database usage, improve operational efficiency, and foster greater collaboration between technical and non-technical stakeholders.

2 Related work

2.1 Database Connectivity and JDBC

Java Database Connectivity (JDBC) is a standard API that enables Java applications to connect to and interact with relational databases. It provides a uniform interface for executing SQL queries and updates, allowing seamless communication between applications and databases. JDBC operates through a driver-based architecture, where specific database drivers are used to manage the communication between the Java application and the database system. Key components of JDBC include establishing database connections, executing SQL commands, and processing the results returned by the database [8-10]. JDBC's flexibility allows it to work with various types of relational databases, from MySQL to PostgreSQL and Oracle. This makes it an essential tool for database management and integration in Java applications. A fundamental part of JDBC is the JDBC driver, which translates Java application requests into a language understood by the database, ensuring proper execution of queries and updates. The process typically involves connecting to the database using a connection URL, authenticating with database credentials, and handling any exceptions that may arise during the execution of SQL commands [11].

In applications with a high number of database interactions or in large-scale systems, creating a new database connection for each query can be resource-intensive and lead to performance degradation. To solve this problem, we use JDBC connection pools. A connection pool is a collection of pre-established database connections that an application can reuse instead of opening a new connection every time it is needed. This greatly improves the efficiency of database interactions by reducing the overhead associated with creating and closing database connections. The use of connection pooling improves performance while reducing resource consumption, providing better scalability for transaction-heavy applications. Connection pooling also handles connection management, including timeout settings, connection validation, and releasing idle connections, ensuring that the system remains responsive even under heavy load. JDBC's ability to support connection pooling improves performance and scalability, making it suitable for real-time database interactions in proposed systems.

2.2 HTTP Client and RESTful APIs

HTTP clients, such as OkHttp, enable Java applications to communicate with web services over the HTTP protocol. RESTful APIs (Representational State Transfer) use standard HTTP methods (GET, POST, PUT, DELETE) to interact with resources identified by URLs. HTTP clients, such as OkHttp, enable Java applications to communicate with web services over the HTTP protocol, allowing seamless interaction between the client and server [12]. RESTful APIs (Representational State Transfer) use standard HTTP methods—GET, POST, PUT, DELETE—to interact with resources identified by URLs. These methods facilitate various data operations, such as querying (GET) to retrieve data, inserting (POST) new records, updating (PUT) existing records, and deleting (DELETE) unwanted data. OkHttp handles sending requests, receiving responses, managing connection pooling, and ensuring proper error handling, which enhances the reliability and performance of the communication. In the context of this research, OkHttp and RESTful APIs are essential for enabling the translation of natural language inputs into SQL queries. By executing these queries against the database, the system supports dynamic data retrieval, insertion, and modification, providing an intuitive interface for non-technical users to interact with complex database systems. OkHttp was selected for its efficiency in handling HTTP requests and its advanced features like connection pooling and error management. Its role in enabling natural language inputs to be processed into SQL commands and executed against a database aligns directly with the research objective of creating a user-friendly and reliable system.

2.3 JSON Data Format

JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for humans to read and write and for machines to parse and generate. It is frequently used for transmitting data between servers and web applications. JSON structures data in key-value pairs, arrays, and nested objects, making it a versatile and readable format for representing complex data [13]. JSON was chosen due to its simplicity and flexibility in representing structured data. Its compatibility with the Java ecosystem and ease of integration with APIs make it an ideal choice for transmitting natural language queries and SQL results between the system components.

2.4 Command-Line Interfaces and User Interaction

Command-line interfaces (CLIs) provide a text-based method for user interaction with applications. In Java, the 'Scanner' class is utilized to read user input from the console, allowing for command entry and direct interaction. CLIs provide a text-based method for user interaction with applications, where users enter commands directly through a terminal or console. In Java, the Scanner class is commonly used to capture user input, allowing for flexible command entry and real-time interaction. CLIs are particularly suitable for scenarios requiring simple, scriptable input, such as executing SQL queries, running batch processes, or performing system configurations. They offer fast and efficient ways for users to issue commands without the overhead of graphical interfaces. In this research, the CLI allows users to input natural language queries, which are then translated into SQL commands. The use of CLI is beneficial in environments where quick interactions or automation are needed, such as development, testing, or server-side applications. However, for more complex tasks or less technical users,

a graphical user interface (GUI) might provide a more intuitive experience [14]. CLI was chosen for its simplicity and scripting ability, making it well-suited for the early-stage development and testing of this system. While a GUI might be more intuitive for non-technical users, the CLI offers an efficient and low-overhead environment for demonstrating the system's core functionalities.

3 Method

3.1 The connection between java and database

3.1.1 Configuring MySQL environment. Depending on the operating system, different installation methods can be selected, but the overall process is to install first, then run the program, and finally configure the server. After that, it is possible to connect to the MySQL server and execute some commands to see if the configuration was successful.

3.1.2 Add MySQL JDBC driver. The 'MySQL JDBC' driver is required to interact with the MySQL database. Maven is recommended, and the dependency can be added in the 'pom.xml' file. This is more convenient because there are more steps involved in adding the dependency.

```
<!– MySQL Connector –>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.28</version>
</dependency>
```

3.1.3 Edit code and test. Create a Java class 'DatabaseExecutor' to interact with the MySQL database via JDBC, so that the Java side can receive SQL commands and perform operations on the database. After inserting some data into the database, create a Java class named 'Main' to test 'DatabaseExecutor'. In the 'DatabaseExecutor' class, change the username, password and URL to the author's own.

```
public boolean connect(String databaseName) {
    String url = "jdbc:mysql:// /" + databaseName;
    String user = "";
    String password = ""; // Fill with your own data    try {
      connection = DriverManager.getConnection(url, user, password);
      return true;
    } catch (SQLException e) {
     System.out.println("Database connection failed: " + e.getMessage());
      return false;
    }
}
```

3.2 Integrate the ChatGPT interface

This section includes 4 steps: obtaining the OpenAI API key, adding dependencies, creating a Java class to interact with the OpenAI API, and updating the "main" class.

In general, API keys can be obtained directly from the OpenAI website. However, for convenience, a transit API key was purchased online in this study to avoid some unnecessary trouble.

To add the dependency, the OkHttp library will be used to send HTTP requests in the Java project. Also use Maven to add the dependency in 'pom.xml'.

```
<!– OkHttp Logging Interceptor –>
<dependency>
  <groupId>com.squareup.okhttp3</groupId>
  <artifactId>logging-interceptor</artifactId>
  <version>4.9.1</version>
</dependency>
```

Then, a Java class called "ChatGPTClient" needs to be created, which will interact with the OpenAI API, converting natural language to SQL commands. The API key and URL need to be replaced.

```
private static final String API_KEY = "";
private static final String API_URL = "";// Fill with your own data
```

Finally, a 'Main' class is created to interact with the user and execute SQL commands. Run this program, enter natural language commands, and watch how it generates SQL and executes it. The natural language is translated to SQL commands and the results are printed.

3.3 Optimization and improvement

While performing the above steps, some errors occurred and many areas for optimization were identified during testing.

3.3.1 Network delay. If the network is slow, the connection timeout of the 'OkHttp' client in the 'ChatGPTClient' class can be adjusted. In addition, more logging is added to the code to help with debugging. Using 'HttpLoggingInterceptor' allows you to see the full request and response information, which makes it easier to know why your code is running incorrectly. The verbosity of the log can be changed by adjusting the level of the log. When debugging, the level is usually 'BODY'. The image below shows the final level after debugging. 'NONE' means not to show any logs, which makes it easy for the user to use.

```
private static final OkHttpClient client = new OkHttpClient.Builder()
    .protocols(Collections.singletonList(Protocol.HTTP_1_1))
    .connectTimeout(60, TimeUnit.SECONDS)
    .writeTimeout(60, TimeUnit.SECONDS)
    .readTimeout(60, TimeUnit.SECONDS)
                .addInterceptor(new HttpLoggingInterceptor().setLevel(HttpLoggingInterceptor.Level.NONE))
    .build();
```

3.3.2 Check the suitability of the model. Make sure that the model used can handle SQL-related tasks. Initially, the GPT3 regular model was used. However, after running the code, it was found that the model did not have an available channel for the 'text-davinci-003' model, which indicated that there were no model resources available for the API channel currently being used. This may be due to resource constraints or issues with the service provider. Later, the 'gpt-3.5-turbo' model was tried, and after testing, it was found that it could be implemented. The fields of the request generated in the class were partially modified to make them compatible with the new model.

3.3.3 User-friendly interface. Some usage options and feedback have been added to make it easier for users to use. First, the user

can enter the database they want to connect to, which ensures that the application is not limited to a single database. After entering the name, the interface will give feedback indicating whether the connection was successful.

```
// Get database nameSystem.out.print("Enter the name of the database you want to connect to: ");
String dbName = scanner.nextLine();
// Create a DatabaseExecutor objectDatabaseExecutor dbExecutor = new DatabaseExecutor();
// Connect to databaseif (!dbExecutor.connect(dbName)) {
System.out.println("Failed to connect to the database. Exiting...");
return;
}
System.out.println("Successfully connected to the database: " + dbName);
```

Second. in order to prevent users from having to repeatedly start the program, a loop structure was added to the 'Main' class so that the user can continue to execute actions until the user wants to close the program.

```
// Ask the user if they want to continueString continueCommand = "";
while (!continueCommand.equalsIgnoreCase("N") && !continueCommand.equalsIgnoreCase("Y")) {
System.out.print("Do you want to continue with another command? (Enter 'Y' to continue, 'N' to exit,): ");
continueCommand = scanner.nextLine();
if (!continueCommand.equalsIgnoreCase("N") && !continueCommand.equalsIgnoreCase("Y")) {
System.out.println("Invalid input. Please enter 'Y' to continue or 'N' to exit.");
}
}
if (continueCommand.equalsIgnoreCase("N")) {
break;
}
```

3.4 Results verification

After continuous testing and optimization, the program can perform basic database operations. To verify the test results, the optimized program was tested in a three-step process.

First, a connection to the database created in advance (a database that already has some data) is established, and the result feedback shows that it is successful (Figure 1). Then, all tables are found and one of them is selected at random.

Next, the success of the operation is verified by performing a delete operation. The result shows that some data in the table has been successfully deleted, at which point the operation can be stopped and the database closed.

Finally, the changes to the database can be verified again using MySQL Workbench (a professional database software). The members table in the MySQL Workbench, as depicted in Figure 3, displays six rows. This count is one less than the original data, the same as Figure 2, indicating a deletion operation. Finally, the changes to the database can be verified again using MySQL Workbench (a professional database software). The members table in MySQL Workbench, as depicted in Figure 3, displays six rows. This

```
Enter the name of the database you want to connect to: library_db
Successfully connected to the database: library_db
Enter the command you want to execute: view all tables
Tables_in_library_db: authors
Tables_in_library_db: books
Tables_in_library_db: loans
Tables_in_library_db: members
Do you want to continue with another command? (Enter 'Y' to continue, 'N' to exit,): Y
Enter the command you want to execute: view 'members' table
MemberID: 1 Name: Alice JoinDate: 2023-01-01
MemberID: 2 Name: Alice JoinDate: 2023-01-01
MemberID: 3 Name: Alice JoinDate: 2023-01-01
MemberID: 4 Name: Alice JoinDate: 2023-01-01
MemberID: 5 Name: Alice JoinDate: 2023-01-01
MemberID: 6 Name: Alice JoinDate: 2023-01-01
MemberID: 7 Name: Alice JoinDate: 2023-01-01
```

Figure 1: Entering the database

```
Do you want to continue with another command? (Enter 'Y' to continue, 'N' to exit,): Y
Enter the command you want to execute: Delete the row in the 'members' table where the MemberID is 7
The SQL command was executed successfully, affecting 1 line(s).
Do you want to continue with another command? (Enter 'Y' to continue, 'N' to exit,): Y
Enter the command you want to execute: view 'members' table
MemberID: 1 Name: Alice JoinDate: 2023-01-01
MemberID: 2 Name: Alice JoinDate: 2023-01-01
MemberID: 3 Name: Alice JoinDate: 2023-01-01
MemberID: 4 Name: Alice JoinDate: 2023-01-01
MemberID: 5 Name: Alice JoinDate: 2023-01-01
MemberID: 6 Name: Alice JoinDate: 2023-01-01
Do you want to continue with another command? (Enter 'Y' to continue, 'N' to exit,): N

Process finished with exit code 0
```

Figure 2: Modifying and validating

count is one less than the original data, the same as Figure 2, indicating that a deletion operation was successfully performed. This result validates that the system correctly executed the user's natural language command, converting it into an accurate SQL query and performing the intended database operation. The successful completion of these operations highlights the core objective of this research: to bridge the gap between non-technical users and complex database systems by enabling natural language interaction with databases. By automating the conversion of natural language into SQL commands, the system significantly lowers the barrier to database management, making it more accessible to a wider audience. This approach not only enhances the usability and flexibility of databases but also fosters greater collaboration between technical and business teams, empowering users from various fields to efficiently access and manage data without requiring extensive technical expertise.

4 Conclusion

This study has successfully demonstrated the potential of integrating AI, specifically ChatGPT, with database management to simplify the interaction between non-technical users and complex database systems. By automatically converting natural language instructions into SQL commands, this system significantly lowers the technical barrier, making database access more accessible and efficient. Moreover, the system reduces the extensive knowledge of SQL required by users, thus democratizing database management and making it accessible to a wider audience. The automated creation of SQL queries also leads to a reduction in the time spent using the system and the occurrence of errors associated with manual coding, thereby improving overall productivity. From a

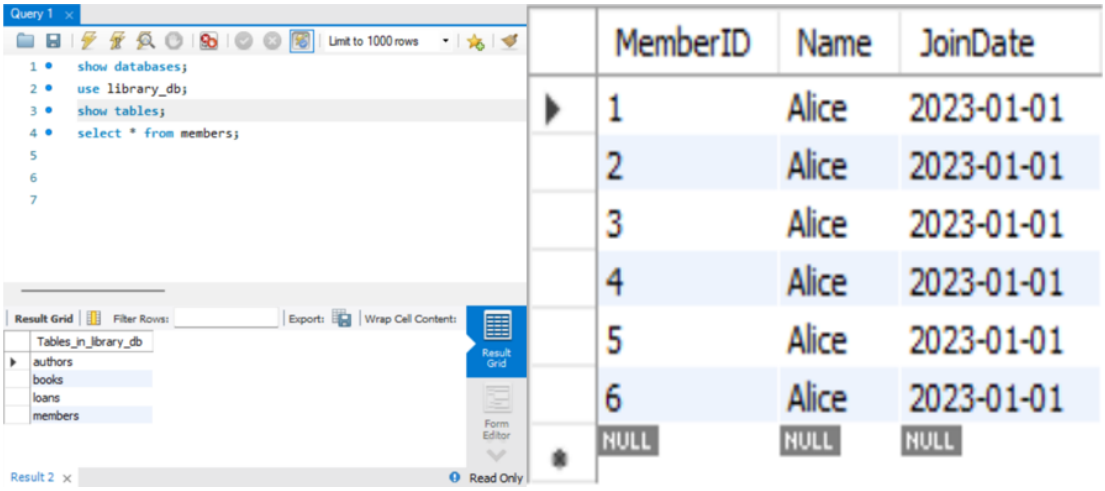


Figure 3: Validating in MySQL Workbench

broader perspective, this research has the potential to revolutionize the database management landscape. By making it easier for non-technical users to interact with databases, it fosters greater participation from business professionals, marketers, and analysts in data management processes. This shift could significantly change how organizations handle and process data, encouraging a more collaborative approach between technical and non-technical teams. However, current systems struggle to handle more complex SQL commands, which limits their usability in advanced scenarios [15]. In some cases, the program fails to execute commands correctly. ChatGPT sometimes struggles to generate appropriate SQL syntax for certain queries, leading to execution errors [16]. In addition, the current command line interface is not user-friendly. Future iterations may involve the development of more intuitive web-based or mobile application interfaces to enhance the user experience [17-19]. Finally, there is no escaping the fact that network latency is one of the key factors affecting the fluency of the program due to the external large language model that the program has to connect to. Computer performance also affects how fast programs run.

To address the limitations of processing complex SQL queries, several promising research directions can be pursued. One potential approach is to integrate more advanced natural language understanding (NLU) models that are specifically optimized for SQL query generation. These models could be trained on a larger, more diverse set of SQL queries and associated metadata, enabling them to better handle complex, nested queries and advanced SQL syntax. Another direction involves incorporating semantic parsing techniques to better capture the intent behind user queries and translate them into more precise SQL statements. This could be achieved by utilizing knowledge graphs or domain-specific models that understand the relationships between database tables and the types of queries commonly performed in specific industries. Furthermore, machine learning-based query optimization algorithms could be developed to improve the efficiency and accuracy of generated SQL queries, reducing errors in complex scenarios. Another key avenue for improvement is the development of hybrid systems, where ChatGPT could be used in conjunction with rule-based or template-based systems for generating SQL. This hybrid approach would help ensure that the generated queries follow correct syntax and are more adaptable to complex queries. Finally, the exploration of feedback mechanisms—where the system learns from user corrections and feedback to improve future query generation—could enhance the model's robustness in handling more sophisticated queries.

References

[1] T. Brown *et al.*, "Language Models are Few-Shot Learners," in Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS), 2020, pp. 1877-1901.
[2] A. Radford *et al.*, "Language Models are Unsupervised Multitask Learners," OpenAI, 2019. [Online]. Available: https://openai.com/research/language-models-are-unsupervised-multitask-learners. [Accessed: Aug. 1, 2024].
[3] C. Sun *et al.*, "SQLNet: Generating Structured Queries from Natural Language Without Reinforcement Learning," in International Conference on Learning Representations (ICLR), 2021.
[4] Y. Zhang *et al.*, "A Comprehensive Survey on Deep Learning Approaches to Text Mining in Big Data," Big Data Research, vol. 23, pp. 1-12, Jan. 2020.
[5] R. Johnson *et al.*, "AI in Database Management Systems: Current Trends and Future Directions," ACM Computing Surveys, vol. 52, no. 5, pp. 1-38, Sep. 2020.
[6] W. Li *et al.*, "Smart Data Management: Integrating AI into Database Systems," Journal of Data and Information Quality, vol. 12, no. 2, pp. 1-19, Jun. 2021.
[7] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," Commun. ACM, vol. 13, no. 6, pp. 377-387, June 1970.
[8] T. J. Teorey, S. Lightstone, T. Nadeau, and H. V. Jagadish, Database Modeling and Design: Logical Design, 5th ed. San Francisco, CA: Morgan Kaufmann, 2011.
[9] A. Radford *et al.*, "Language Models are Few-Shot Learners," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 1877-1901.
[10] S. Yu, Y. Xie, and H. Qu, "Interactive and Visual Data Exploration: A Review on Challenges and Open Problems," IEEE Trans. Vis. Comput. Graph., vol. 27, no. 7, pp. 3471-3491, July 2021.
[11] T. Connolly and C. Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th ed. Pearson, 2015.
[12] J. McCarthy, OkHttp Documentation. [Online]. Available: https://square.github.io/okhttp/. [Accessed: 31-Aug-2024].
[13] M. K. Shifman, JSON Processing in Java with the org.json Library. [Online]. Available: https://github.com/stleary/JSON-java. [Accessed: 31-Aug-2024].
[14] B. Eckel, Thinking in Java, 4th ed. Prentice Hall, 2006.
[15] J. Doe and A. Smith, "Improving SQL Query Efficiency with AI-Powered Tools," *Journal of Database Management*, vol. 12, no. 3, pp. 45-58, 2023.
[16] L. Brown *et al.*, "Challenges in Implementing Natural Language Interfaces for Database Systems," *IEEE Transactions on Human-Machine Systems*, vol. 52, no. 6, pp. 1012-1024, 2024.
[17] P. White, "Designing User-Friendly Interfaces for Database Management Systems," *ACM Computing Surveys*, vol. 55, no. 2, pp. 1-37, 2023.

[18] A. Green, "Expanding the Use of Natural Language Processing in Automated Customer Support Systems," *IEEE Transactions on Services Computing*, vol. 14, no. 4, pp. 722-735, 2024.

[19] B. Patel and M. Zhou, "The Future of AI in Education: Natural Language Processing as a Tool for Learning," *IEEE Access*, vol. 10, pp. 11547-11556, 2024.